Manual técnico y de usuario del prototipo basado en la aplicación de aprendizaje profundo para estimar el tiempo de vida útil restante en motores turbofan de aeronave comercial Boeing 777

Trabajo Terminal No. 2027-A190

*Alumnos: *Rojas Vasquez Evelyne*[1]*, Onofre Rosales Rodrigo*[2]
Directores: Morales Guitron Sandra Luz[1]*, López Ruiz Gabriela de Jesús*[2]
**e-mail: erojasv2200@alumno.ipn.mx*

Ficha del documento	
Documento	Manual técnico y de usuario (entregable de la fase de Documentación del cronograma)
Versión	1.0 — octubre de 2026
Ubicación en el repositorio	`docs/manual_usuario/manual_usuario.tex` (fuente) y `docs/manual_usuario/manual_usuario.pdf` (compilado)
Corresponde a	Código del repositorio versión 1.0.0 (`src/__init__.py`)
Dirigido a	Operadores del departamento de mantenimiento aeronáutico (uso del tablero) y personal técnico (instalación, entrenamiento y evaluación)

Índice

1. Introducción **3**
1.1. Alcance y advertencia sobre los datos . . . 3
1.2. Qué encontrará en este manual . . . 3

2. Descripción general del sistema **3**
2.1. Metodología aplicada . . . 4

3. Requisitos **5**

4. Instalación paso a paso **5**
4.1. Paso 1. Obtener el código . . . 5
4.2. Paso 2. Crear y activar un entorno virtual . . . 5
4.3. Paso 3. Instalar las dependencias . . . 6
4.4. Paso 4. Verificar la instalación . . . 6

5. Obtención y descripción de los datos **6**
5.1. Descarga . . . 6
5.2. Subconjuntos y archivos . . . 7
5.3. Tratamiento automático de los datos . . . 7

6. Entrenamiento y evaluación del modelo **8**
6.1. Entrenar un subconjunto . . . 8
6.2. Arquitectura e hiperparámetros por defecto . . . 8
6.3. Búsqueda en cuadrícula de hiperparámetros . . . 9
6.4. Artefactos generados . . . 9
6.5. Interpretación de las métricas . . . 10

7. Uso del tablero de visualización **10**
7.1. Iniciar el tablero . . . 10
7.2. Distribución de la pantalla . . . 11
7.3. Procedimiento de uso . . . 12
7.4. Sistema de alertas . . . 12
7.5. Cómo interpretar la curva de RUL . . . 12
7.6. Actualizar el tablero después de reentrenar . . . 13

8. Estructura del repositorio **13**
8.1. Compilar este manual . . . 13

9. Solución de problemas **14**

10. Limitaciones y uso responsable **14**

11. Glosario **15**

1. Introducción

Este documento explica cómo instalar, ejecutar y utilizar el **prototipo** de sistema de mantenimiento predictivo desarrollado en el Trabajo Terminal No. 2027-A190. El prototipo **aplica** una arquitectura de red neuronal recurrente LSTM reportada en la literatura [8] para estimar la vida útil restante (*Remaining Useful Life*, RUL) de motores turbofan a partir de lecturas multivariadas de sensores, y presenta los resultados en un tablero de visualización con un indicador de alerta de falla.

Siguiendo la terminología del protocolo, el término **prototipo** designa el sistema integrado (módulos de software, flujo de datos y tablero), mientras que el término **modelo** designa la red LSTM que funciona como núcleo de inferencia.

1.1 Alcance y advertencia sobre los datos

El caso de aplicación es el motor turbofan de aeronaves comerciales como el Boeing 777. Sin embargo, el prototipo se entrena y valida con los conjuntos de datos públicos C-MAPSS de la NASA [5], que son resultado de una **simulación** de un motor turbofan genérico [1]. Por lo tanto:

- Los datos **no** son telemetría real de motores Boeing 777.
- Los canales disponibles son temperaturas, presiones, velocidades, razones y flujos (Tabla 7); **no** existe un canal de vibración.
- Los resultados son de carácter académico y **no** sustituyen los procedimientos de mantenimiento ni de aeronavegabilidad vigentes.

1.2 Qué encontrará en este manual

Tabla 1: Guía de lectura según el perfil del usuario.

Perfil	**Qué necesita hacer**	**Secciones**
Operador de mantenimiento	Consultar el estado de un motor o de la flota e interpretar las alertas	7, 10
Personal técnico / desarrollador	Instalar, obtener los datos, entrenar, evaluar y mantener el sistema	3–6, 8, 9
Evaluador (sinodal)	Reproducir los resultados y verificar la metodología	2, 6, 8

2. Descripción general del sistema

El prototipo se organiza en cuatro módulos que operan en secuencia, tal como se describe en el protocolo. La Figura 1 muestra el flujo completo: a la izquierda, la **entrada** (datos crudos de sensores del motor); a la derecha, la **salida** (valor estimado de RUL en ciclos de operación y alerta visual en el tablero).

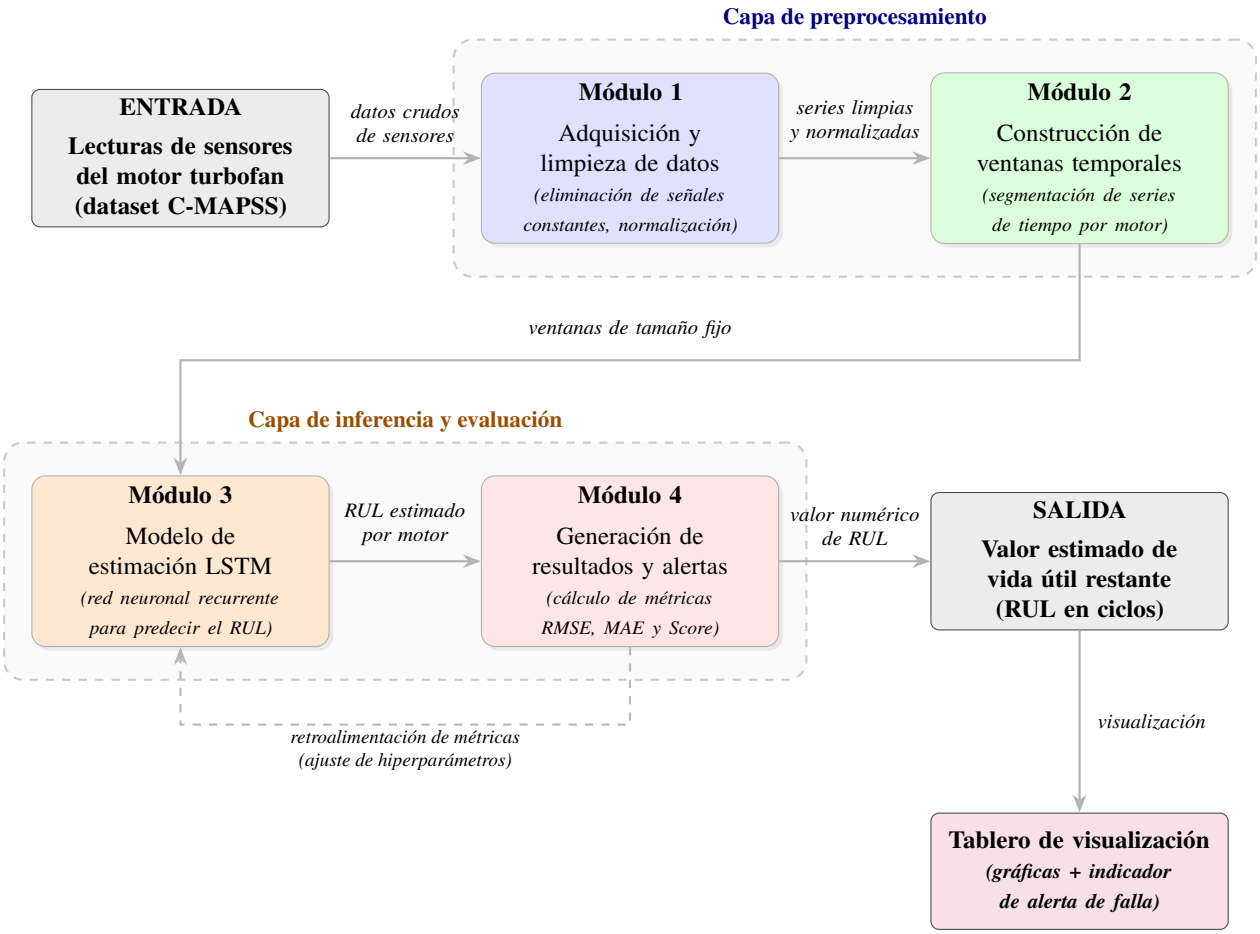


Figura 1: Arquitectura modular del sistema de estimación de vida útil restante (RUL). La **entrada** son los datos de sensores del motor; la **salida** es el número de ciclos de operación restantes estimados por el modelo. El flujo se lee de izquierda a derecha en cada fila y continúa en la fila inferior.

La Tabla 2 indica en qué archivos del repositorio se implementa cada módulo.

Tabla 2: Correspondencia entre los módulos del protocolo y el código fuente.

Módulo	**Archivos**	**Responsabilidad**
1. Adquisición y limpieza	`src/ingestion.py`, `src/preprocessing.py`	Descarga y lectura validada de C-MAPSS; eliminación de sensores constantes; normalización; etiquetas de RUL
2. Ventanas temporales	`src/preprocessing.py`	Ventanas deslizantes por motor; partición entrenamiento/validación por motor
3. Modelo LSTM	`src/model.py`, `src/train.py`	Arquitectura LSTM; optimizador Adam con reducción adaptativa de la tasa de aprendizaje; entrenamiento
4. Resultados y alertas	`src/evaluation.py`, `src/inference.py`, `dashboard/app.py`	Métricas RMSE, MAE y Score; gráficas; curva de RUL; alertas; tablero

2.1 Metodología aplicada

El desarrollo sigue el estándar CRISP-DM [7]. Las seis fases se agrupan en cuatro etapas ejecutables que se reflejan en el uso del sistema (Tabla 3).

Tabla 3: Etapas del proyecto, fases CRISP-DM y comando asociado.

Etapa	Fases CRISP-DM	Dónde se ejecuta
1. Recopilación y preparación de datos	1, 2 y 3	`make data`; cuaderno `notebooks/01_EDA.ipynb`
2. Adaptación y entrenamiento del modelo	4	`make train`, `make grid`
3. Evaluación y validación	5	Evaluación automática al final de `make train`; `make evaluate`
4. Despliegue: tablero de visualización	6	`make dashboard`

3. Requisitos

Tabla 4: Requisitos de hardware y software.

Componente	Requisito
Sistema operativo	Windows 10/11, macOS o Linux
Python	Versión 3.10 a 3.12 (con `pip` y `venv`)
Memoria RAM	8 GB recomendados (el modelo es pequeño; el entrenamiento es viable sin GPU)
Disco	Aproximadamente 3 GB para dependencias (TensorFlow) y datos
Bibliotecas principales	TensorFlow $\geq$ 2.16 (Keras 3), NumPy, Pandas, Scikit-Learn, Matplotlib, Plotly y Streamlit $\geq$ 1.50; la lista completa está en `requirements.txt`
Herramientas opcionales	`make` (en Windows puede sustituirse, ver Tabla 5); Git; LaTeX (`latexmk`) para compilar este manual
Conexión a Internet	Necesaria para instalar dependencias y descargar C-MAPSS

4. Instalación paso a paso

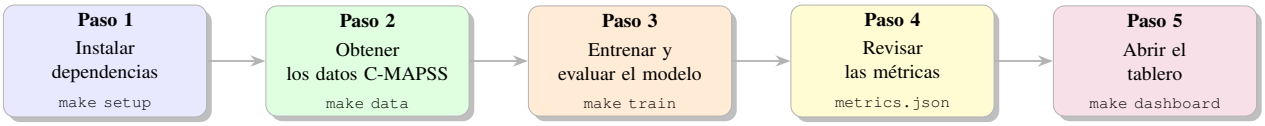


Figura 2: Flujo de uso del prototipo, desde la instalación hasta el tablero.

4.1 Paso 1. Obtener el código

```
git clone <URL_DEL_REPOSITORIO> rul-turbofan-tt
cd rul-turbofan-tt
```

Todos los comandos de este manual se ejecutan desde la **raíz del repositorio** (la carpeta que contiene `Makefile` y `README.md`).

4.2 Paso 2. Crear y activar un entorno virtual

Linux o macOS:

```
python -m venv .venv
source .venv/bin/activate
```

Windows (PowerShell):

```
python -m venv .venv
.venv\Scripts\activate
```

El indicador del entorno (`(.venv)`) debe aparecer al inicio de la línea de comandos.

4.3 Paso 3. Instalar las dependencias

```
make setup
```

Este comando ejecuta `python -m pip install -r requirements-dev.txt`, que incluye las dependencias de ejecución, *pytest*, *ruff* y JupyterLab. La instalación de TensorFlow puede tardar varios minutos.

4.4 Paso 4. Verificar la instalación

```
make test
```

Deben ejecutarse 24 pruebas unitarias y de extremo a extremo (usan datos sintéticos, por lo que no requieren descargar C-MAPSS). La salida correcta termina con una línea similar a `24 passed`.

Tabla 5: Equivalencia entre comandos `make` y comandos directos (útil en Windows sin `make`).

Con make	Comando directo
`make setup`	`python -m pip install -r requirements-dev.txt`
`make data`	`python -m src.ingestion --download`
`make test`	`python -m pytest -q`
`make train SUBSET=FD001 WINDOW=30`	`python -m src.train --subset FD001 --window 30 --epochs 100`
`make grid SUBSET=FD001`	`python -m src.grid_search --subset FD001 --epochs 100`
`make evaluate SUBSET=FD001`	`python -m src.evaluation --subset FD001`
`make dashboard`	`streamlit run dashboard/app.py`
`make manual`	`cd docs/manual_usuario` y luego `latexmk -pdf manual_usuario.tex`

5. Obtención y descripción de los datos

5.1 Descarga

```
make data
```

El comando descarga el archivo comprimido de C-MAPSS, lo extrae en `data/raw/` y verifica que existan los 12 archivos esperados. Si la descarga falla (la ubicación del recurso en los servidores de la NASA ha cambiado en el pasado), hay dos alternativas:

1. Descargar manualmente el conjunto desde el repositorio oficial de datos de pronóstico de la NASA [5] y copiar los archivos `train_FD00X.txt`, `test_FD00X.txt` y `RUL_FD00X.txt` (con X = 1, 2, 3, 4) en `data/raw/`.
2. Indicar la dirección vigente: `python -m src.ingestion --download --url <URL>`.

Para comprobar la carga, ejecute `python -m src.ingestion`: imprime el número de motores y de filas de cada subconjunto.

5.2 Subconjuntos y archivos

Tabla 6: Subconjuntos de C-MAPSS [5].

Subconjunto	Regímenes de operación	Modos de falla	Motores de entrenamiento	Motores de prueba	
FD001	1	1	100	100	
FD002	6	1	260	259	
FD003	1	2	100	100	
FD004	6	2	249	248	

Cada subconjunto consta de tres archivos de texto:

- `train_FD00X.txt`: trayectorias completas de cada motor, desde el inicio de la operación hasta la falla.
- `test_FD00X.txt`: trayectorias truncadas antes de la falla.
- `RUL_FD00X.txt`: RUL real del último ciclo observado de cada motor de prueba (una línea por motor).

Cada línea de `train_` y `test_` contiene 26 columnas separadas por espacios: identificador del motor (`unit`), ciclo (`cycle`), tres ajustes operacionales (`op1`–`op3`) y 21 canales de sensores (`s1`–`s21`).

Tabla 7: Canales de sensores de C-MAPSS [1, 5].

Canal	Símbolo	Descripción
s1	T2	Temperatura total a la entrada del fan
s2	T24	Temperatura total a la salida del compresor de baja (LPC)
s3	T30	Temperatura total a la salida del compresor de alta (HPC)
s4	T50	Temperatura total a la salida de la turbina de baja (LPT)
s5	P2	Presión a la entrada del fan
s6	P15	Presión total en el ducto de bypass
s7	P30	Presión total a la salida del HPC
s8	Nf	Velocidad física del fan
s9	Nc	Velocidad física del núcleo (compresor)
s10	epr	Razón de presión del motor
s11	Ps30	Presión estática a la salida del HPC
s12	phi	Razón entre el flujo de combustible y Ps30
s13	NRf	Velocidad corregida del fan
s14	NRc	Velocidad corregida del núcleo
s15	BPR	Razón de bypass
s16	farB	Razón combustible/aire del quemador
s17	htBleed	Entalpía de sangrado
s18	Nf_dmd	Velocidad del fan demandada
s19	PCNfR_dmd	Velocidad corregida del fan demandada
s20	W31	Sangrado de enfriamiento de la turbina de alta (HPT)
s21	W32	Sangrado de enfriamiento de la turbina de baja (LPT)

5.3 Tratamiento automático de los datos

Al entrenar, el sistema realiza de forma automática las siguientes operaciones (Módulos 1 y 2):

1. **Partición por motor:** el 20 % de los motores de entrenamiento se reserva para validación interna. La partición se hace por motor, no por ventana, para evitar que ventanas casi idénticas de un mismo motor queden en ambos conjuntos.

2. **Eliminación de señales constantes:** se descarta un sensor si su desviación estándar dentro de cada régimen de operación es menor que 0.01 en todos los regímenes. La lista de sensores conservados y descartados se guarda en `metrics.json`.
3. **Normalización por régimen de operación:** escalamiento mínimo–máximo con estadísticas calculadas *solo* con los motores de entrenamiento. En FD002 y FD004 los seis regímenes se identifican con K-Means sobre los ajustes operacionales.
4. **Etiqueta de RUL lineal por tramos:** $\mathrm{RUL}(t) = t_{\text{final}} - t$, limitada a un tope (por defecto 125 ciclos) [2].
5. **Ventanas deslizantes:** ventanas de longitud fija (por defecto 30 ciclos, paso 1); la etiqueta es la RUL del último ciclo de la ventana. En prueba se usa la última ventana de cada motor; si la trayectoria es más corta que la ventana, se rellena replicando la primera lectura.

6. Entrenamiento y evaluación del modelo

6.1 Entrenar un subconjunto

```
make train SUBSET=FD001 WINDOW=30
```

Para entrenar los cuatro subconjuntos con las ventanas predeterminadas (30, 20, 30 y 20 ciclos para FD001 a FD004, respectivamente):

```
make train-all
```

El proceso ejecuta, en orden: carga y limpieza, construcción de ventanas, entrenamiento con parada temprana, evaluación en el conjunto de prueba, y guardado de modelo, métricas y gráficas.

6.2 Arquitectura e hiperparámetros por defecto

El modelo es una red LSTM apilada seguida de capas densas y una salida lineal que entrega la RUL en ciclos [8, 3]. El entrenamiento usa el optimizador Adam [4] con tasa de aprendizaje inicial 10^{-3}, reducción adaptativa de la tasa al detectarse estancamiento de la pérdida de validación (*ReduceLROnPlateau*) y parada temprana. Para prevenir el sobreajuste se emplea abandono (*dropout*) [6].

Tabla 8: Parámetros de `python -m src.train`.

Parámetro	Valor por defecto	Descripción
`--subset`	FD001	Subconjunto: FD001, FD002, FD003 o FD004
`--window`	30	Longitud de la ventana temporal (ciclos)
`--rul-cap`	125	Tope de la RUL lineal por tramos
`--val-fraction`	0.2	Fracción de motores para validación interna
`--lstm-units`	64 32	Celdas de cada capa LSTM (una cifra por capa)
`--dense-units`	32 16	Neuronas de cada capa densa
`--dropout`	0.2	Tasa de abandono
`--learning-rate`	0.001	Tasa de aprendizaje inicial de Adam
`--epochs`	100	Máximo de épocas (la parada temprana puede detener antes)
`--batch-size`	256	Tamaño de lote
`--seed`	42	Semilla de reproducibilidad
`--raw-dir`	`data/raw`	Carpeta con los archivos de C-MAPSS
`--out-dir`	`models`	Carpeta de salida de los artefactos
`--verbose`	1	0 = silencioso; 1 = muestra el progreso por época

Ejemplo con hiperparámetros personalizados:

```
python -m src.train --subset FD002 --window 20 --lstm-units 128 64 --dropout 0.3 --epochs 150
```

Nota. Los valores por defecto son un punto de partida, no los valores finales; deben refinarse con la búsqueda en cuadrícula y contrastarse con lo reportado en la literatura [8]. La ubicación de datos y modelos también puede cambiarse con las variables de entorno `RUL_DATA_DIR` y `RUL_MODELS_DIR`.

6.3 Búsqueda en cuadrícula de hiperparámetros

```
make grid SUBSET=FD001
```

Explora combinaciones de tamaño de ventana, celdas por capa LSTM y tasa de abandono. **La selección se basa únicamente en la pérdida de la validación interna**: el conjunto de prueba no interviene, para no sobreajustar la selección a la evaluación final. Por defecto se evalúan 4 ventanas $\times$ 4 configuraciones LSTM $\times$ 3 tasas de abandono = 48 entrenamientos, lo que puede tardar horas; para una exploración más breve:

```
python -m src.grid_search --subset FD001 --windows 20 30 --lstm-options 32,32 64,32 --dropouts
    0.1 0.2 --epochs 40
```

El resultado se guarda en `models/grid_search/grid_FD001.csv`, ordenado de menor a mayor `val_rmse`; la primera fila es la mejor configuración. Con ella se vuelve a ejecutar `src.train` para el entrenamiento final.

6.4 Artefactos generados

Cada entrenamiento crea la carpeta `models/<SUBCONJUNTO>/`:

Tabla 9: Archivos generados por el entrenamiento.

Archivo	**Contenido**
`model.keras`	Modelo LSTM entrenado (devuelve RUL en ciclos)
`preprocessor.joblib`	Sensores seleccionados, regímenes y parámetros de normalización
`config.json`	Hiperparámetros del experimento
`metrics.json`	Métricas de validación y de prueba; sensores usados y descartados
`history.csv`	Pérdida y métricas por época
`test_predictions.csv`	RUL real y estimada de cada motor de prueba
`figures/pred_vs_real.png`	RUL real vs. estimada (motores ordenados por RUL real) y dispersión
`figures/error_dist.png`	Histograma del error de estimación
`figures/training_history.png`	Curvas de pérdida y de la tasa de aprendizaje

Para reevaluar un modelo ya entrenado sobre el conjunto de prueba:

```
make evaluate SUBSET=FD001
```

6.5 Interpretación de las métricas

Sea $d_i = \widehat{\mathrm{RUL}}_i - \mathrm{RUL}_i$ el error del motor i (RUL estimada menos RUL real) y N el número de motores de prueba. Se calculan las siguientes métricas [5]:

$$\mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} d_i^2}, \qquad \mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N} |d_i|, \tag{1}$$

$$\mathrm{Score} = \sum_{i=1}^{N} s_i, \qquad s_i = \begin{cases} e^{-d_i/13} - 1, & d_i < 0 \quad \text{(estimación temprana)} \\ e^{d_i/10} - 1, & d_i \geq 0 \quad \text{(estimación tardía)} \end{cases} \tag{2}$$

- **RMSE** y **MAE** se expresan en ciclos; menor es mejor. El RMSE pondera más los errores grandes.
- **Score** es una función de puntuación asimétrica; menor es mejor. Penaliza más las estimaciones **tardías** ($d_i > 0$, la RUL se *sobreestima*), porque implican operar el motor más allá de su vida segura, que las tempranas ($d_i < 0$).
- `pct_estimacion_tardia` indica el porcentaje de motores con $d_i > 0$; un valor muy superior a 50 % señala un modelo sesgado hacia el optimismo.

En `metrics.json` se reportan tres bloques: `validation_windows` (todas las ventanas de los motores de validación), `test` (última ventana de cada motor de prueba contra la RUL real *sin tope*) y `test_capped_labels` (igual, pero con la RUL real limitada al tope). Al comparar con otros trabajos, indique cuál convención utiliza, porque la literatura no es uniforme.

7. Uso del tablero de visualización

7.1 Iniciar el tablero

Requisito: haber entrenado al menos un subconjunto (Sección 6) y contar con los datos en `data/raw/`.

```
make dashboard
```

El navegador se abre en `http://localhost:8501`. Si no se abre automáticamente, copie esa dirección en el navegador. Para usar otro puerto: `streamlit run dashboard/app.py --server.port 8502`. Para detener el tablero, presione `Ctrl+C` en la terminal.

Si no hay modelos entrenados, el tablero lo indica con un mensaje y el comando que debe ejecutarse (`make train SUBSET=FD001`).

7.2 Distribución de la pantalla

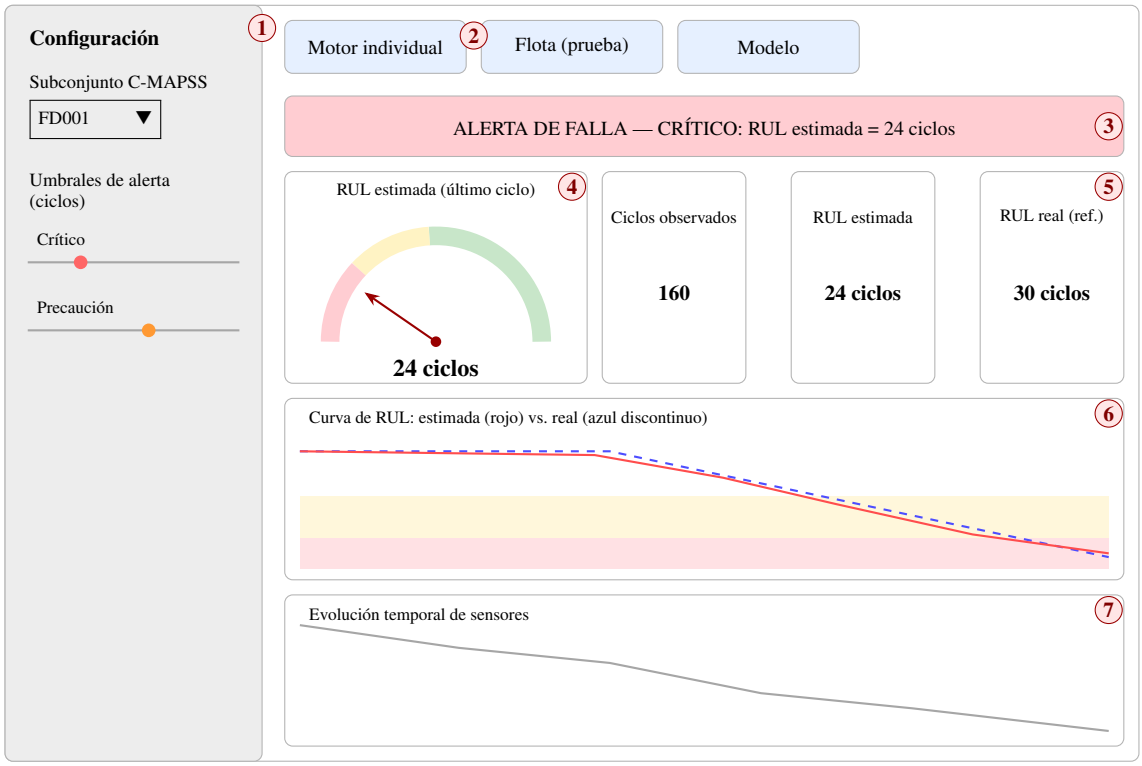


Figura 3: Esquema de la pestaña «Motor individual». Los números se explican en la Tabla 10. La figura es ilustrativa; los valores mostrados son de ejemplo.

Tabla 10: Elementos de la pestaña «Motor individual» (Figura 3).

N.º	Elemento	Función
1	Barra lateral	Selecciona el subconjunto C-MAPSS (solo aparecen los entrenados) y ajusta los umbrales de alerta
2	Pestañas	«Motor individual», «Flota (prueba)» y «Modelo»
3	Banner de alerta	Estado del motor: NORMAL (verde), PRECAUCIÓN (amarillo) o CRÍTICO (rojo)
4	Indicador tipo velocímetro	RUL estimada en el último ciclo observado, con las zonas de color de los umbrales
5	Indicadores numéricos	Ciclos observados, RUL estimada y RUL real de referencia con su error (estimada − real)
6	Curva de RUL	RUL estimada y RUL real en cada ciclo; bandas de color de los umbrales; zona gris de ventana incompleta
7	Sensores	Evolución temporal de los sensores elegidos, en valores originales o normalizados

7.3 Procedimiento de uso

1. **Elegir el subconjunto.** En la barra lateral, seleccione FD001, FD002, FD003 o FD004. Solo se listan los subconjuntos con un modelo entrenado en `models/`.
2. **Ajustar los umbrales (opcional).** Con los deslizadores defina los ciclos restantes a partir de los cuales el estado pasa a PRECAUCIÓN y a CRÍTICO (Sección 7.4).
3. **Seleccionar un motor** en la pestaña «Motor individual». El tablero calcula la RUL estimada en cada ciclo y muestra el estado actual.
4. **Leer la alerta y el indicador** (elementos 3 y 4 de la Figura 3).
5. **Analizar la curva de RUL.** Compare la estimación con la RUL real y observe la tendencia.
6. **Explorar los sensores.** Elija uno o varios sensores en la lista (se muestra el símbolo y la descripción). Active «Mostrar valores normalizados» para compararlos en una escala común por régimen.
7. **Revisar la flota.** En «Flota (prueba)» se listan todos los motores ordenados por RUL estimada ascendente, de modo que los de intervención más urgente aparecen primero; el gráfico de barras y la tabla se actualizan con los umbrales elegidos.
8. **Consultar el modelo.** En «Modelo» se muestran RMSE, MAE y Score de prueba, los sensores utilizados y descartados, los hiperparámetros y la gráfica de RUL real vs. estimada.

7.4 Sistema de alertas

El estado se determina comparando la RUL estimada del último ciclo con dos umbrales configurables (Figura 4 y Tabla 11).

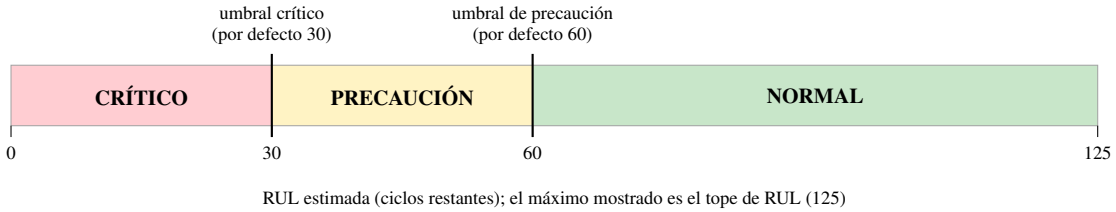


Figura 4: Zonas de alerta según la RUL estimada, con los umbrales por defecto.

Tabla 11: Estados de alerta del tablero.

Estado	Condición	Acción sugerida
NORMAL	RUL estimada $>$ umbral de precaución	Operación normal; seguimiento rutinario
PRECAUCIÓN	Umbral crítico $<$ RUL estimada $\leq$ umbral de precaución	Planificar el mantenimiento y reservar recursos
CRÍTICO	RUL estimada $\leq$ umbral crítico	Programar la intervención en la próxima escala planificada

Los umbrales por defecto (60 y 30 ciclos) no provienen de la literatura: son parámetros de política de mantenimiento que debe definir el departamento usuario según sus costos, tiempos de reposición y escalas disponibles.

7.5 Cómo interpretar la curva de RUL

- Al inicio de la vida del motor, tanto la RUL real como la estimada se muestran limitadas al tope (125 ciclos): se asume que mientras el motor está sano la degradación es despreciable [2].

- La zona gris «ventana incompleta» corresponde a los primeros ciclos, en los que la ventana de entrada se completa replicando la primera lectura; las estimaciones ahí son **menos confiables**.
- Es más informativa la **tendencia descendente sostenida** que un valor puntual: una estimación aislada puede oscilar por ruido.
- La RUL real de referencia solo existe para los motores del conjunto de prueba de C-MAPSS; con datos reales de operación no estaría disponible.
- Un error positivo (estimada $>$ real) es una estimación **tardía** y es el caso de mayor riesgo operativo.

7.6 Actualizar el tablero después de reentrenar

El tablero guarda en memoria el modelo de cada subconjunto. Después de volver a entrenar, deténgalo (`Ctrl+C`) y vuelva a iniciarlo, o use el menú *Clear cache* de Streamlit y recargue la página.

8. Estructura del repositorio

```
rul-turbofan-tt/
|-- data/
|   |-- raw/                  # C-MAPSS original (no versionado)
|   `-- processed/            # artefactos intermedios (no versionado)
|-- docs/
|   `-- manual_usuario/       # este manual (.tex, .pdf, figuras/)
|-- notebooks/
|   |-- 01_EDA.ipynb          # analisis exploratorio
|   `-- 02_experimentacion.ipynb
|-- src/                      # modulos del prototipo
|   |-- config.py  ingestion.py  preprocessing.py
|   |-- model.py  train.py  grid_search.py
|   `-- evaluation.py  inference.py
|-- models/<FD00X>/           # modelo, preprocesador, metricas y figuras
|-- dashboard/app.py          # tablero Streamlit
|-- tests/                    # pruebas con pytest (24)
|-- Makefile  README.md  requirements*.txt  pytest.ini  ruff.toml
`-- .gitignore
```

8.1 Compilar este manual

```
make manual
```

Requiere una distribución LaTeX con `latexmk` y los paquetes *tikz*, *babel*, *diagbox*, *listings* y *makecell* (incluidos en TeX Live y MiKTeX completos). El resultado es `docs/manual_usuario/manual_usuario.pdf`. Para incluir capturas reales, guárdelas como `figuras/tablero_motor.png` y `figuras/tablero_flota.png` dentro de la carpeta del manual; se insertarán automáticamente al compilar.

9. Solución de problemas

Tabla 12: Problemas frecuentes y su solución.

Síntoma	Causa probable	Solución
`FileNotFoundError: Faltan [...] en data/raw`	Los datos no se han descargado o están en otra carpeta	Ejecute `make data` o copie los archivos manualmente (Sección 5); verifique `RUL_DATA_DIR`
La descarga con `make data` falla	La URL de la NASA cambió o no hay conexión	Descargue el conjunto manualmente o use `--url` con la dirección vigente
`make: command not found` (Windows)	`make` no está instalado	Use los comandos directos de la Tabla 5
`ModuleNotFoundError: No module named 'src'`	El comando se ejecutó fuera de la raíz del repositorio	Ubíquese en la carpeta que contiene `Makefile` y use `python -m src.train`
Error al importar TensorFlow o Keras	Versión de Python no compatible o instalación incompleta	Use Python 3.10–3.12 en un entorno virtual limpio y reinstale con `make setup`
El tablero muestra «No se encontraron modelos entrenados»	No se ha entrenado ningún subconjunto	Ejecute `make train SUBSET=FD001` y recargue la página
El puerto 8501 está ocupado	Hay otra instancia de Streamlit en ejecución	Use `--server.port 8502` o cierre la instancia anterior
El tablero no refleja un modelo recién entrenado	Caché en memoria del tablero	Reinicie el tablero o use *Clear cache*
Memoria insuficiente al entrenar	Lote o modelo demasiado grandes para el equipo	Reduzca `--batch-size` (p. ej., 64) o las celdas de `--lstm-units`
RMSE muy alto o la pérdida no disminuye	Hiperparámetros poco adecuados o pocas épocas	Revise `figures/training_history.png`; ejecute `make grid`; aumente `--epochs`
La búsqueda en cuadrícula tarda demasiado	Se evalúan 48 combinaciones por defecto	Reduzca las listas de `--windows`, `--lstm-options` y `--dropouts`, y las épocas
Resultados distintos entre ejecuciones	Semilla distinta o diferencias de hardware	Fije `--seed`; pequeñas diferencias numéricas entre equipos son esperables

10. Limitaciones y uso responsable

- **Datos simulados.** Los resultados corresponden a C-MAPSS y no garantizan el desempeño sobre flotas reales ni sobre un modelo de aeronave específico.
- **Prototipo académico.** El sistema es una herramienta de apoyo a la decisión y de demostración; no está certificado ni sustituye las inspecciones y los programas de mantenimiento aprobados.
- **Umbrales de alerta no calibrados** con costos operativos reales (Sección 7.4).
- **Incertidumbre.** La estimación es puntual; el sistema no reporta intervalos de confianza.
- **Fidelidad de las comparaciones.** Los resultados deben compararse con otros trabajos solo si comparten el protocolo de evaluación (tope de RUL, tamaño de ventana, uso del conjunto de prueba).

11. Glosario

Tabla 13: Términos utilizados en este manual.

Término	**Definición**
C-MAPSS	*Commercial Modular Aero-Propulsion System Simulation*: simulador y conjuntos de datos de degradación de motores turbofan de la NASA
Ciclo	Una unidad de operación del motor (por ejemplo, un vuelo) en el que se registran las lecturas de los sensores
RUL	*Remaining Useful Life*: ciclos de operación que restan antes de la falla
Tope de RUL	Valor máximo de la RUL usada como etiqueta (RUL lineal por tramos); por defecto 125 ciclos
Régimen de operación	Combinación de condiciones de vuelo (ajustes operacionales); FD002 y FD004 tienen seis
Ventana temporal	Secuencia de los últimos W ciclos de lecturas que recibe la red para estimar la RUL
LSTM	*Long Short-Term Memory*: red neuronal recurrente capaz de aprender dependencias de largo plazo [3]
Dropout (abandono)	Técnica de regularización que desactiva neuronas al azar durante el entrenamiento para reducir el sobreajuste [6]
Validación interna	20 % de los motores de entrenamiento reservados para seleccionar hiperparámetros y detener el entrenamiento
Estimación tardía / temprana	Error positivo ($\widehat{\text{RUL}} > \text{RUL}$) o negativo, respectivamente
RMSE, MAE, Score	Métricas de desempeño definidas en la Sección 6
CRISP-DM	Proceso estándar de la industria para proyectos de minería de datos [7]

Referencias

[1] Frederick, D., de Castro, J. y Litt, J. (2007). *User's Guide for the Commercial Modular Aero-Propulsion System Simulation (C-MAPSS)*. NASA Technical Memorandum NASA/TM–2007-215026. Glenn Research Center, Cleveland, OH: NASA.

[2] Heimes, F. O. (2008). Recurrent neural networks for remaining useful life estimation. En *2008 International Conference on Prognostics and Health Management (PHM08)* (pp. 1–6). Denver, CO: IEEE.

[3] Hochreiter, S. y Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

[4] Kingma, D. P. y Ba, J. (2015). Adam: A method for stochastic optimization. En *3rd International Conference on Learning Representations (ICLR 2015)*. San Diego, CA. https://arxiv.org/abs/1412.6980

[5] Saxena, A., Goebel, K., Simon, D. y Eklund, N. (2008). Damage propagation modeling for aircraft engine run-to-failure simulation. En *Proceedings of the 1st International Conference on Prognostics and Health Management (PHM08)*. Denver, CO: IEEE. https://doi.org/10.1109/PHM.2008.4711414

[6] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. y Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56), 1929–1958.

[7] Wirth, R. y Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. En *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*, Manchester, 29–39.

[8] Zheng, S., Ristovski, K., Farahat, A. y Gupta, C. (2017). Long short-term memory network for remaining useful life estimation. En *2017 IEEE International Conference on Prognostics and Health Management (ICPHM)* (pp. 88–95). Dallas, TX: IEEE.